TP 5
From a mesh to a sparse linear system

This TP opens the second part of the semester, devoted to linear systems $Ax = b$. We start from the heated plate seen in the lecture: we model it by a linear system built on a mesh, store this system in a sparse format, and implement the product of its matrix with a vector. The four exercises follow these steps: build the mesh (Exercise 1), find the neighbours of each vertex (Exercise 2), build A and b (Exercise 3), multiply A by a vector (Exercise 4). Solving the system is the topic of the next TPs, and several of the solvers will only use this product.

How to work. Everything is in the single file `tp5.c`. It contains:

- *Part 0* (provided): the mesh structure of TP 2 (`struct Mesh2D`) and the edge table of TP 3 (`struct EdgeTable`), together with the boundary temperature `g`. You do not need your own code of TP 2 and TP 3: everything you need is recalled in this sheet;
- *Parts 1 to 4*: one part per exercise below, with the functions to complete, marked `TODO`;
- *Part 5* (provided): functions which print what you have built;
- the function `main`, empty for now. In each exercise, you add to `main` (before the line `return 0;`) the lines given in the sheet: they build the example of the figure below and print it, and the sheet shows what you should see.

To compile and run, type in a terminal, in the folder containing `tp5.c` (in VS Code: *Terminal* → *New Terminal*):

```
gcc -Wall tp5.c -o tp5
./tp5
```

The option `-Wall` asks the compiler to warn about suspicious code: read the warnings, they often point to a bug. Recompile and run after each function you write. The useful terminal commands are summarised in the separate cheat sheet.

The heated plate

A thin rectangular metal plate $[0, L] \times [0, H]$, with $L = 2$ and $H = 1$, is heated on one side and cooled on the other: its left edge is kept at temperature 100, its right edge at 0, and along the top and bottom edges the temperature decreases linearly from 100 to 0. In other words, the temperature on the boundary is

$$g(x, y) = 100 \left(1 - \frac{x}{L}\right).$$

Heat flows from warm to cold regions until the temperature no longer changes. We want this steady temperature inside the plate.

We mesh the plate with triangles and compute the temperature only at the vertices; two vertices are *neighbours* if they are joined by an edge. At a boundary vertex w, the temperature $g(w)$ is known. At an interior vertex v, the temperature x_v is unknown and, in the steady state, equals

the average of the temperatures at its neighbours (otherwise heat would flow to or from v):[1]

$$x_v = \frac{1}{\deg(v)} \sum_{w \text{ neighbour of } v} (\text{temperature at } w),$$

where $\deg(v)$, the *degree* of v, is its number of neighbours. The diagonals of our mesh alternate (see the figure below), so that the neighbours of each vertex are spread evenly in all directions.

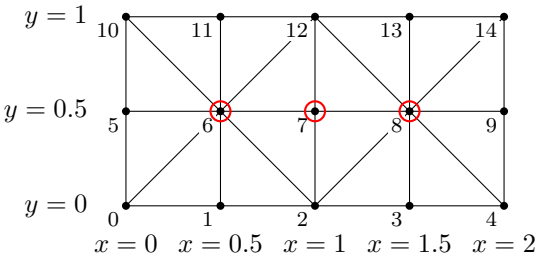
Multiplying by $\deg(v)$, and moving the known temperatures to the right-hand side, we get one linear equation per interior vertex:

$$\deg(v)\, x_v \; - \sum_{\substack{w \text{ interior} \\ \text{neighbour of } v}} x_w \; = \sum_{\substack{w \text{ boundary} \\ \text{neighbour of } v}} g(w).$$

Numbering the interior vertices $0, 1, \ldots, N-1$, these N equations form a linear system $Ax = b$, with $A \in \mathbb{R}^{N \times N}$ and $b \in \mathbb{R}^N$.

Example. Below, the plate is cut into $n_x = 4$ cells in the x-direction and $n_y = 2$ cells in the y-direction, and each cell into two triangles. The interior vertices 6, 7, 8 (in red) carry the unknowns x_0, x_1, x_2. Vertex 6 has degree 8; its neighbours are 7 (interior) and $0, 1, 2, 5, 10, 11, 12$ (boundary), with temperatures $g = 100, 75, 50, 100, 100, 75, 50$. Its equation is therefore $8x_0 - x_1 = 550$. Vertex 7 has degree 4; its neighbours are 6 and 8 (interior) and 2, 12 (boundary, $g = 50$), so that $4x_1 - x_0 - x_2 = 100$. Doing the same for vertex 8 gives

$$\begin{pmatrix} 8 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 8 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 550 \\ 100 \\ 150 \end{pmatrix}, \qquad \text{whose solution is } (x_0, x_1, x_2) = (75, 50, 25).$$



You will check your code on this example.

Exercises

Exercise 1. The mesh of the plate. Complete

```
int plate_mesh2D(struct Mesh2D *m, int nx, int ny);
```

which builds the mesh of $[0, L] \times [0, H]$ with $n_x \times n_y$ cells, as in the figure:

- the mesh has $(n_x + 1)(n_y + 1)$ vertices and $2n_x n_y$ triangles;
- vertex (i, j), for $0 \le i \le n_x$ and $0 \le j \le n_y$, has index $j(n_x + 1) + i$ and coordinates $(iL/n_x,\, jH/n_y)$;

[1] This is the mean value property of the solutions of Laplace's equation $\partial_{xx} u + \partial_{yy} u = 0$, which describes the steady temperature.

- the cell with lower left vertex (i, j) has vertices $v_{00} = j(n_x+1)+i$ (lower left), $v_{10} = v_{00}+1$ (lower right), $v_{01} = v_{00} + n_x + 1$ (upper left) and $v_{11} = v_{01} + 1$ (upper right). It is cut along a diagonal which alternates from one cell to the next:
  - if $i + j$ is even, into the triangles (v_{00}, v_{10}, v_{11}) and (v_{00}, v_{11}, v_{01});
  - if $i + j$ is odd, into the triangles (v_{00}, v_{10}, v_{01}) and (v_{10}, v_{11}, v_{01}).

  In all cases, the vertices of each triangle are listed counterclockwise.

Recall (Part 0): `initialize_mesh2D(m, nv, nt)` allocates the two arrays of the mesh; `m->vert[v].x` and `m->vert[v].y` are the coordinates of vertex `v`, and `m->tri[t].v[0]`, `m->tri[t].v[1]`, `m->tri[t].v[2]` are the vertices of triangle `t`. To number the triangles, use a counter `t`, increased by one after each triangle. Careful with integer division in C: since `i` and `nx` are integers, `i / nx * L` gives 0 whenever $i < n_x$. Write `i * L / nx` instead, where `L` is a `double`.

To check your function on the example, add to `main`:

```
struct Mesh2D m;
plate_mesh2D(&m, 4, 2);
print_mesh(&m);
```

The program prints the 15 vertices and the 16 triangles. Compare them with the figure. For instance, you should see

```
  vertex  7 = (1, 0.5)
  ...
  triangle  0 = (0, 1, 6)
  triangle  1 = (0, 6, 5)
  triangle  2 = (1, 2, 6)
  triangle  3 = (2, 7, 6)
```

If gnuplot is installed, add the line `mesh2D_to_gnuplot(&m, "mesh.dat");` and look at the mesh with `gnuplot -p -e "plot 'mesh.dat' with lines"`.

Exercise 2. Boundary vertices and neighbours. Recall the edge table of TP 3: edge j of triangle i is the oriented edge from `tri[i].v[j]` to `tri[i].v[(j+1)%3]`, and its code is $3i + j$. All the edges starting at a vertex `v` are visited by the loop

```
for (int e = et->head[v]; e != -1; e = et->next[e]) {
  int i = e / 3, j = e % 3;   /* e is in triangle i, and v = m->tri[i].v[j] */
}
```

Each triangle containing `v` has exactly one edge starting at `v`, so this loop visits all the triangles containing `v`.

1) Complete `boundary_vertices`, which returns an array of `m->nv` integers, equal to 1 for the boundary vertices and to 0 for the interior ones. An edge $a \to b$ lies on the boundary if and only if the opposite edge $b \to a$ does not exist, i.e. `edge_table_find(b, a, et, m)` returns -1; then a and b are boundary vertices. Start with `int *on_bdry = calloc(m->nv, sizeof(int));`, which gives an array filled with zeros, then loop over the triangles `i` and over `j` $= 0, 1, 2$: the edges of the mesh are the edges from `a = m->tri[i].v[j]` to `b = m->tri[i].v[(j+1)%3]`.

2) Complete `vertex_neighbours`, which writes the neighbours of `v` into the array `nbrs`, and returns their number, i.e. the degree of `v`. The neighbours of `v` are the other vertices of the triangles containing `v`: in triangle `i` of the loop above, these are `m->tri[i].v[(j+1)%3]` and

`m->tri[i].v[(j+2)%3]`. A neighbour may belong to two of these triangles: before adding a vertex `w` to `nbrs`, check with a loop that it is not already there.

3) Add to `main`:

```
struct EdgeTable et = build_edge_table(&m);
int *on_bdry = boundary_vertices(&m, &et);
print_neighbours(&m, &et, on_bdry);
```

For each vertex, the program prints whether it lies on the boundary, and its neighbours. Only vertices 6, 7, 8 should be interior. You should see, for instance,

```
  vertex  0 (boundary): 3 neighbours: 6 5 1
  vertex  6 (interior): 8 neighbours: 12 11 7 10 5 2 1 0
  vertex  7 (interior): 4 neighbours: 8 12 6 2
```

Compare with the figure (the order of the neighbours does not matter).

Exercise 3. The linear system in CRS format. On our plate meshes, each row of A has at most 9 nonzero entries (the diagonal and at most 8 neighbours), however fine the mesh. We therefore only store the nonzero entries, in the Compressed Row Storage format seen in the lecture (CRS, also called CSR):

```
struct CSRMatrix {
  int n;        /* the matrix is n x n */
  int nnz;      /* number of stored entries */
  double *val;  /* size nnz: the values, row after row */
  int *col_ind; /* size nnz: the column of each value */
  int *row_ptr; /* size n+1: row i is stored at positions
                   row_ptr[i], ..., row_ptr[i+1]-1 */
};
```

In each row, we store the diagonal entry first; the order of the other entries does not matter. For the 3×3 matrix of the example, one possible storage is

$$\texttt{val} = [\,8,-1,\ \ 4,-1,-1,\ \ 8,-1\,], \quad \texttt{col_ind} = [\,0,1,\ \ 1,0,2,\ \ 2,1\,], \quad \texttt{row_ptr} = [\,0,2,5,7\,].$$

The functions `csr_initialize`, `csr_dispose` and `csr_print` are provided. In the functions below, to get the neighbours of a vertex `v`, declare an array `int nbrs[MAX_NBRS];` and call `int d = vertex_neighbours(v, et, m, nbrs);`: then `d` is the degree of `v`, and its neighbours are `nbrs[0]`, ..., `nbrs[d-1]`.

1) Complete `number_unknowns`, which sets `num[v]` to -1 for a boundary vertex, numbers the interior vertices $0, 1, \ldots, N-1$ in increasing order, and returns N.

2) Complete `count_nonzeros`, which returns the number of nonzero entries of A: for each interior vertex, 1 (the diagonal entry) plus the number of its interior neighbours.

3) Complete `assemble_plate`. The array `b` is allocated by the caller, with `N` entries. The function proceeds as follows.

   - Allocate A with `csr_initialize(A, N, count_nonzeros(m, et, num))`.
   - Set `A->row_ptr[0] = 0`, and use a variable `pos`, initialised to 0, for the next free position in `val` and `col_ind`.
   - Loop over the vertices. For an interior vertex `v`, with `i = num[v]`, write row i: first the diagonal entry $\deg(v)$, in column i; then -1 in column `num[w]` for each interior neighbour `w`. Set `b[i] = 0`, then add the temperature `g(m->vert[w].x, m->vert[w].y)` of each boundary neighbour `w`. At the end of the row, set `A->row_ptr[i+1] = pos`.

Since `num` increases with `v`, the rows are written in the order $0, 1, \ldots, N-1$, so that `row_ptr` is filled from left to right.

4) Add to `main` (the sizes 15 and 3 are those of the example):

```
int num[15];
int N = number_unknowns(on_bdry, m.nv, num);
print_numbering(num, m.nv);
printf("  N = %d, nnz = %d\n", N, count_nonzeros(&m, &et, num));
struct CSRMatrix A;
double b[3];
assemble_plate(&m, &et, num, N, &A, b);
print_system(&A, b);
```

You should see the numbering of the vertices, the three arrays of the CRS format, and the system of the example:

```
num = -1 -1 -1 -1 -1 -1 0 1 2 -1 -1 -1 -1 -1 -1
N = 3, nnz = 7
...
as a full matrix, with b on the right:
     8   -1    0   | 550
    -1    4   -1   | 100
     0   -1    8   | 150
```

The order of the entries within a row may differ, but the full matrix and `b` must be these.

5) For $n_x = 2000$ and $n_y = 1000$, what is N? How much memory is needed to store A as a full $N \times N$ array of `double` (8 bytes each)? And in CRS format (`int`: 4 bytes; use nnz $\leq 9N$)? If you compute these numbers in C, use `double`: N^2 does not fit in an `int`.

Exercise 4. Matrix–vector product.

1) Complete `csr_matvec`, which computes $y = Ax$, i.e. $y_i = \sum_j a_{ij} x_j$, using only the stored entries: for each row i, sum `val[k] * x[col_ind[k]]` over the positions k of row i. How many multiplications does it perform? How many would a product with the full $N \times N$ matrix need?

2) Add to `main`:

```
double ones[3] = {1, 1, 1}, y[3];
csr_matvec(&A, ones, y);
print_vector("A * (1, 1, 1)", y, 3);
```

You should get $(7, 2, 7)$. Explain why $(Ax)_i$, for $x = (1, \ldots, 1)$, is the number of boundary neighbours of interior vertex i.

At the end of `main`, release the memory:

```
free(on_bdry); csr_dispose(&A); dispose_edge_table(&et); dispose_mesh2D(&m);
```

To remember

The matrix A is *large* (about two million rows for $n_x = 2000$, $n_y = 1000$), *sparse* (at most 9 nonzero entries per row), *symmetric* and, as we will see, *positive definite*. The next TPs solve $Ax = b$ with direct, then iterative methods; the latter only need the product $x \mapsto Ax$.